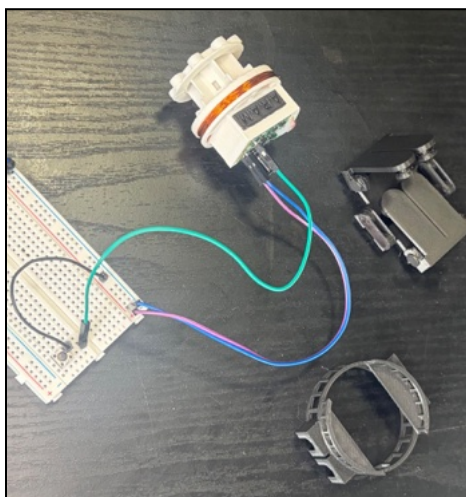
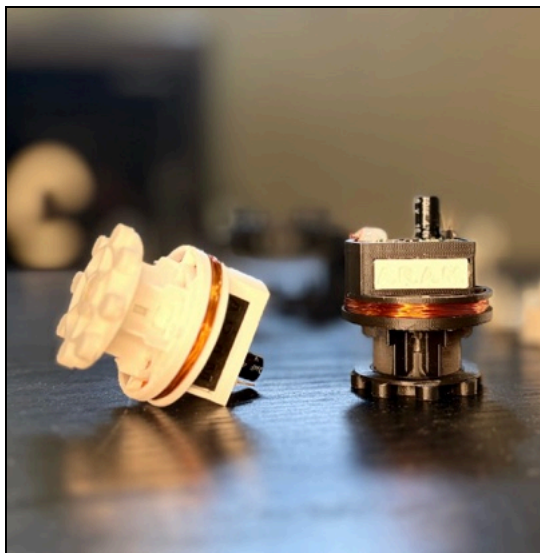




Metal Orchestra Quick Start Guide



Product of American Robotics Assisted Manufacturing

www.ARAM-USA.com

View further kit usage information & legal information



Table Of Contents

Table Of Contents & Introduction	1
Pinout Diagram	2
Getting Started With The Arduino Library: Basic Usage	3 - 5
Making Instruments: Creating Your First Instrument	6
Making Instruments: Basic Bluetooth MIDI Instrument	7, 8
Making Instruments: Optimized Bluetooth MIDI instrument	9 - 11
MIDI Bluetooth Operation: Using With a Mac	12 - 14
MIDI Bluetooth Operation: Using With a Windows Computer	15 -19
Tuning: How to Tune Your Actuator to Almost Any Resonator	20, 21
Conclusion: Thank You, Contact Information, Legal Disclaimer	22

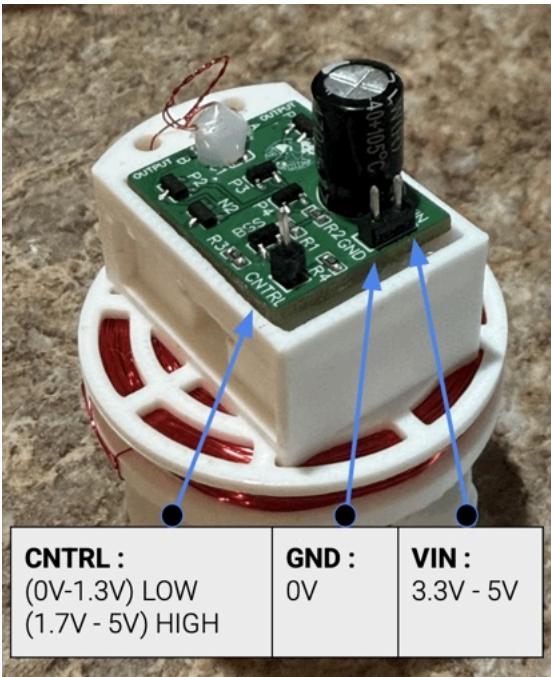
Introduction

Thanks for using ARAM's Metal Orchestra Kit. Refer to the ARAM-MOTCA_0_10-0SP **Spec Sheet** and ARAM-MOTCA_0_10-0SP **Quick Start Guide** for basic usage. This document contains information on how to create and use a **Bluetooth MIDI instrument** from the kit.

*Please read the full spec sheet and view legal disclaimers.



Pinout and Basic Usage



Pin	Voltage	Additional Comments
VIN	3.3V to 5V Continuous 5V+ to 12V Pulsed	Incorrectly plugging VIN to 0V may damage your unit
GND	0V	Incorrectly plugging GND to voltage may damage your unit
CNTRL	(0V - 1.3V) Low (1.7V - 5V) High	(Default) Pulldown to 0V <i>High Signal</i> will cause strike <i>Low Signal</i> will cause retract

Getting Started With The Arduino Library: Basic Usage

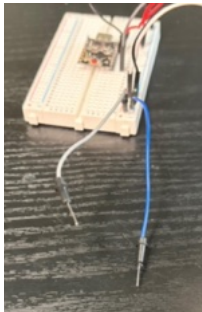
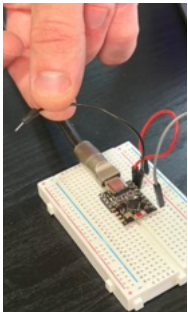
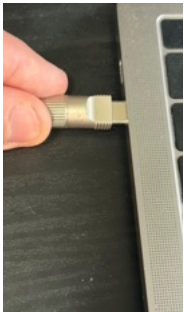
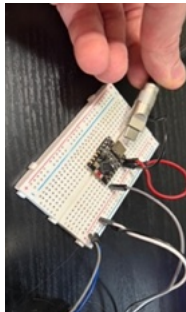
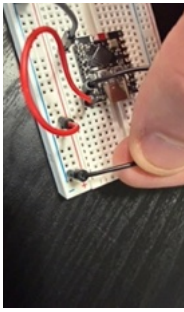
Before Using The Software



ARAM-MOTCA_0_10-0SP Quick Start Guide

Please make sure the actuator is correctly wired according to the pinout and example circuits in the ARAM-MOTCA_0_10-0SP Quick Start Guide. **Incorrectly switching polarity will damage the unit, (incorrect wiring of Vin and GND).**

When Uploading Code Please **Disconnect** The **Power Supply** and the **Actuator**
This avoids drawing too much power from your computer during programming, or additional issues with ports.

					
Step 1: Disconnect power supply	Step 2: Disconnect power wire for actuator	Step 3: Connect Esp32 to PC using usb-c You're ready to upload your code!	Step 4: Disconnect Usb-c when upload is complete	Step 5: Reconnect power wire for actuator	Step 6: Reconnect power supply It's ready to be used now!

When attaching multiple actuators to an instrument ensure the power draw does not exceed the maximum of the power supply. The starter kit comes with a 2 amp power supply, which can supply enough power for 3 - 4 actuators and an ESP32.

Using the Software

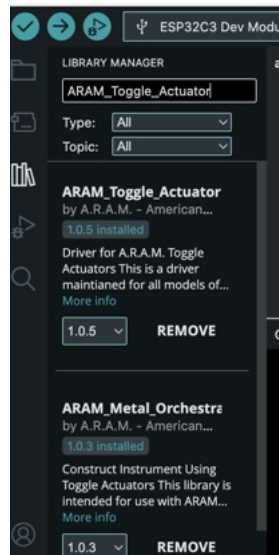
First let's make sure you have the right software, you'll need the **Arduino IDE**



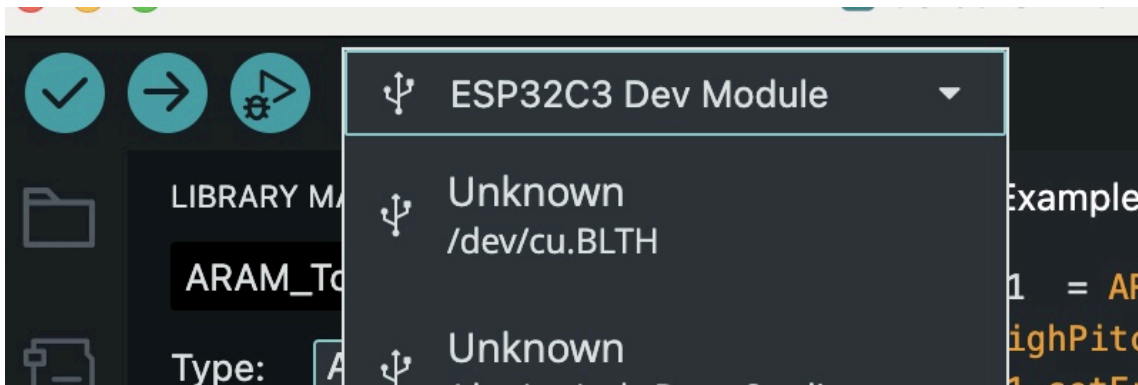
ARAM-MOTCA_0_10-0SP *Quick Start Guide*

<https://www.arduino.cc/en/software/>

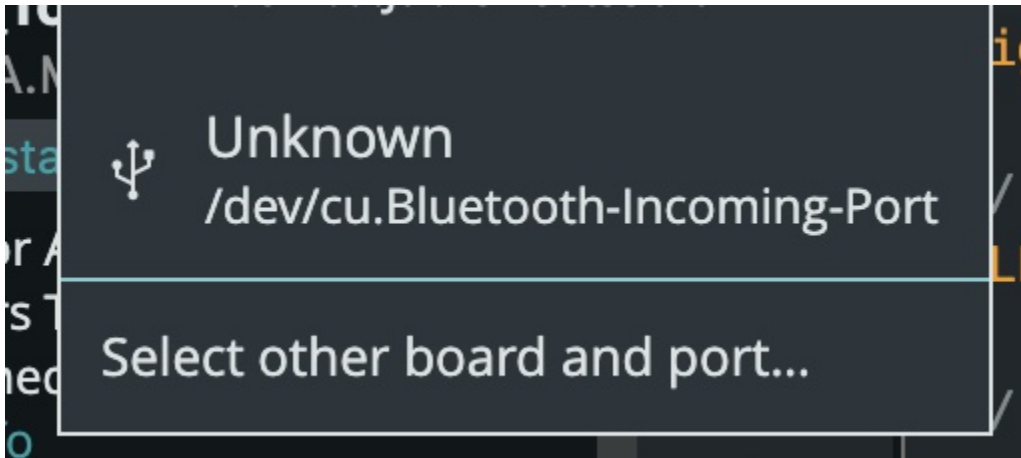
Next to use the actuator you'll need either the `ARAM_Toggle_Actuator` or the `ARAM_Metal_Orchestra` library. Search for either by name in the **Library Manager**



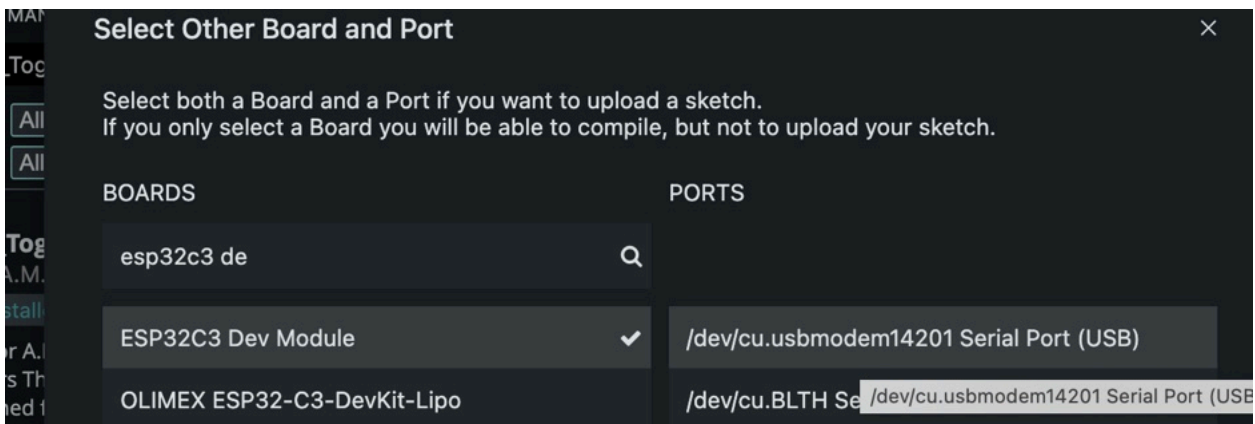
Next you'll need to import whichever library you downloaded into your program, see any of the above code examples, or click more info on either of the libraries to view code examples



Before uploading your code, ensure that the correct output is selected.



If you don't see *ESP32C3 Dev Module* appear automatically, select the "**Select other board and port...**"



Search and Select **ESP32C3 Dev Module** in the boards section
Select the **usb Serial Port** in the ports section



Making Instruments: Creating your first instrument

This example code shows creating an instrument, then playing C4.

```
//import ARAM Metal Orchestra Library
#include <ARAM_Metal_Orchestra.h>

//define what pin we're gonna use
#define CNTRL_PIN_A 1 //change me to your desired cntrl pin A

//declare toggle actuator
ARAM_TOGGLE_ACTUATOR a1;

//declare Instrument
ARAM_METAL_ORCHESTRA_INSTRUMENT instrument;

void setup() {
  //initialize toggle actuator with preferred frequency of 800 hz
  a1 = ARAM_TOGGLE_ACTUATOR(CNTRL_PIN_A, 800);

  //initialize toggle actuator with preferred frequency of 800 hz
  instrument.setInstrumentMode(ARAM_METAL_ORCHESTRA_Resonator);

  //add actuator to instrument
  instrument.addToggleActuator(a1);

  //Cause the actuator to play the note C4
  instrument.handleNoteOn(0, 60, 127, 0);
  //Cause the actuator to stop playing the note C4
  //instrument.handleNoteOff(0,60,0,0)
}

void loop() {
  //in the loop we must call update on the instrument
  instrument.update();
}
```

You can add up to 6 actuators in the same manner, you'll notice that this time the actuator is declared with a preferred frequency. This is for use with the instrument in choosing which actuator to play specific incoming notes on when the instrument has multiple actuators.

The instrument will accept basic MIDI noteOn and noteOff commands:

`instrument.handleNoteOn( channel, frequency, velocity, timestamp)`

This is inline with the MIDI note standard.

For all cases timestamp can be ignored, depending on the specific settings velocity and channel are generally ignored.



Making Instruments: Basic Bluetooth MIDI Instrument

This example code shows creating a bluetooth MIDI instrument

```
//import ARAM Metal Orchestra Library and Bluetooth MIDI library
#include <ARAM_Metal_Orchestra.h>
#include <BLEMidi.h>

//define what pin we're gonna use
#define CNTRL_PIN_A 1 //change me to your desired cntrl pin A

//declare toggle actuator
ARAM_TOGGLE_ACTUATOR a1;

//declare Instrument
ARAM_METAL_ORCHESTRA_INSTRUMENT instrument;

void handleNoteOn(uint8_t channel, uint8_t pitch, uint8_t velocity, uint16_t timestamp) {
    instrument.handleNoteOn(channel, pitch, velocity, timestamp);
}

void handleNoteOff(uint8_t channel, uint8_t pitch, uint8_t velocity, uint16_t timestamp) {
    instrument.handleNoteOff(channel, pitch, velocity, timestamp);
}

void setup() {
    // Initialize a toggle actuator on pin_a, with preferred frequency of 800hz
    a1 = ARAM_TOGGLE_ACTUATOR(CNTRL_PIN_A, 800);

    // add toggle actuator to instrument
    instrument.addToggleActuator(a1);

    // ensure instrument is in standard operating mode
    instrument.setInstrumentMode(ARAM_METAL_ORCHESTRA_Resonator);

    // Initialize the built-in BLEMidiServer object
    BLEMidiServer.begin("A.R.A.M. Metal Orchestra");

    // Register callbacks to the global object
    BLEMidiServer.setNoteOnCallback(handleNoteOn);
    BLEMidiServer.setNoteOffCallback(handleNoteOff);
}
```


ARAM-MOTCA_0_10-0SP **Quick Start Guide**

```
void loop() {  
  // call update during the loop on the instrument  
  instrument.update();  
}
```

This is your standard setup for a bluetooth midi instrument, the code is largely the same compared to the basic instrument code. You will however notice:

BLEMidiServer.begin

This creates a bluetooth midi server

Along with

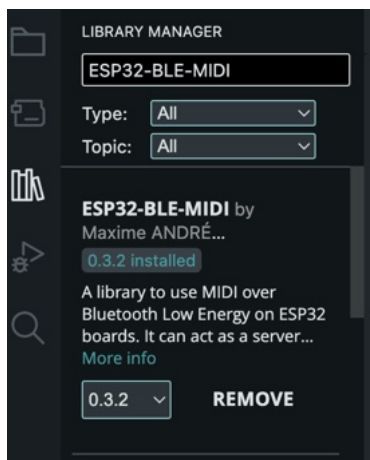
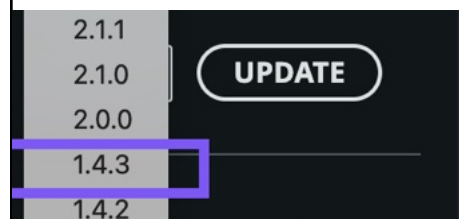
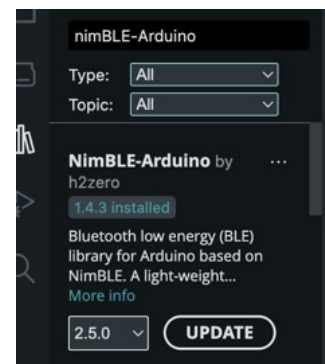
BLEMidiServer.setNoteOnCallback

BLEMidiServer.setNoteOffCallback

These define what functions the server sends the notes to

Important:

this requires the additional download of the BLE MIDI

Install ESP32-BLE-MIDI Library**Select nimBLE-Arduino in the library manager: Downgrade to 1.4.3**



Making Instruments: Optimized Bluetooth MIDI Instrument

This example code shows creating a bluetooth midi instrument with a queue to avoid blocking.

```
//import ARAM Metal Orchestra Library and Bluetooth MIDI library
#include <ARAM_Metal_Orchestra.h>
#include <BLEMidi.h>

//define what pin we're gonna use
#define CNTRL_PIN_A 1 //change me to your desired cntrl pin A

//declare toggle actuator
ARAM_TOGGLE_ACTUATOR a1;

//declare Instrument
ARAM_METAL_ORCHESTRA_INSTRUMENT instrument;

//declare MidiEvent object, we use this to represent requested notes
struct MidiEvent {
    bool isNoteOn;
    uint8_t channel;
    uint8_t pitch;
    uint8_t velocity;
};

//here we actually define the queue in memory, with a maximum size of 16 queued events
#define MIDI_QUEUE_SIZE 16
volatile MidiEvent midiQueue[MIDI_QUEUE_SIZE];
volatile uint8_t queueHead = 0;
volatile uint8_t queueTail = 0;

//this functions adds the MidiEvent to the queue
void pushMidiEvent(bool isNoteOn, uint8_t channel, uint8_t pitch, uint8_t velocity) {
    uint8_t nextHead = (queueHead + 1) % MIDI_QUEUE_SIZE;
    if (nextHead != queueTail) {
        midiQueue[queueHead].isNoteOn = isNoteOn;
        midiQueue[queueHead].channel = channel;
        midiQueue[queueHead].pitch = pitch;
        midiQueue[queueHead].velocity = velocity;
        queueHead = nextHead;
    }
}
```



```
//these two functions tell the esp32 to add the midi events to the queue when they arrive
instead of playing them immediately
void handleNoteOn(uint8_t channel, uint8_t pitch, uint8_t velocity, uint16_t timestamp) {
    pushMidiEvent(true, channel, pitch, velocity);
}

void handleNoteOff(uint8_t channel, uint8_t pitch, uint8_t velocity, uint16_t timestamp) {
    pushMidiEvent(false, channel, pitch, velocity);
}

void setup() {
    // Initialize a toggle actuator on pin_a, with preferred frequency of 800hz
    a1 = ARAM_TOGGLE_ACTUATOR(CNTRL_PIN_A,800);

    // add toggle actuator to instrument
    instrument.addToggleActuator(a1);

    // ensure instrument is in standard operating mode
    instrument.setInstrumentMode(ARAM_METAL_ORCHESTRA_Resonator);

    // Initialize the built-in BLEMidiServer object
    BLEMidiServer.begin("A.R.A.M. Metal Orchestra");

    // Register callbacks to the global object
    BLEMidiServer.setNoteOnCallback(handleNoteOn);
    BLEMidiServer.setNoteOffCallback(handleNoteOff);
}
```



```
void loop() {
    // call update during the loop on the instrument
    instrument.update();

    // Process the queued MIDI events safely on Core 1
    if (queueTail != queueHead) {

        //grab midiEvent from queue
        MidiEvent event;
        event.isNoteOn = midiQueue[queueTail].isNoteOn;
        event.channel = midiQueue[queueTail].channel;
        event.pitch = midiQueue[queueTail].pitch;
        event.velocity = midiQueue[queueTail].velocity;
        if(event.velocity==0)event.isNoteOn = false;

        //delete the midiEvent from the queue so we don't play it twice
        queueTail = (queueTail + 1) % MIDI_QUEUE_SIZE;

        //this is a helper function that allows you to convert a midi pitch into a frequency (hz)
        double frequency = midiNoteToFrequency(event.pitch);

        //Now we play the note
        if (event.isNoteOn) {
            highPitchInstrument.handleNoteOn(event.channel, event.pitch, event.velocity, 0);
        } else {
            highPitchInstrument.handleNoteOff(event.channel, event.pitch, event.velocity, 0);
        }
    }
}
```

*The big change here is the inclusion of a queue. This is important to stop **Blocking**, when an esp32 receives a bluetooth command it is through something called an **interrupt**. If the esp32 is still in the middle of an interrupt when a second bluetooth command comes in, it may skip.*

By adding a queue we make sure the interrupt is as short as possible, and we can actually play the note when the interrupt is over and regular operation has resumed.

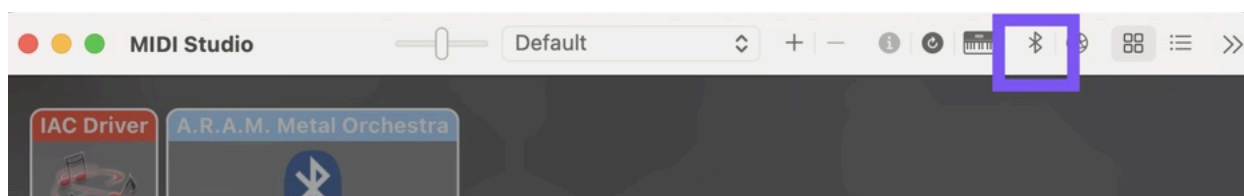


MIDI Bluetooth Operation: Using With a Mac

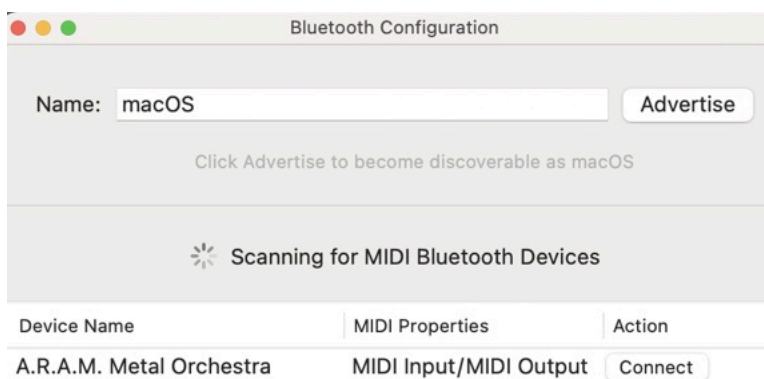
Recommended Software

Software	Audio MIDI Setup	Pocket MIDI	Aria Maestosa
Download	Included on mac	https://apps.apple.com/us/app/pocket-midi/id1260936756?mt=12	https://ariamaestosa.github.io/ariamaestosa/docs/index.html
Use	Connect to your instrument	Play individual notes	Play sheet music

Connecting to your instrument



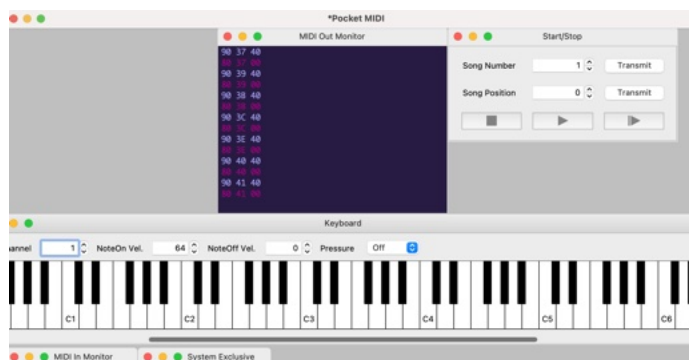
Open “Audio MIDI Setup”, select the bluetooth icon in the corner



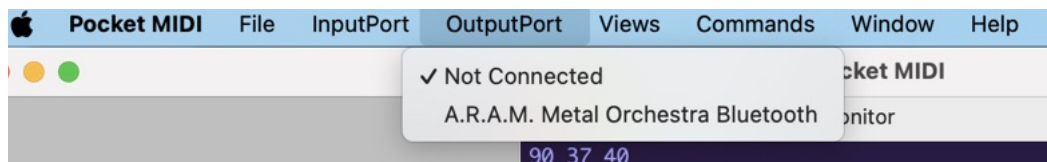
In the popup window find the device named “A.R.A.M. Metal Orchestra” and select connect

You’re now successfully connected to your Metal Orchestra

**If the device does not appear, try resetting the ESP32 with the reset button*

ARAM-MOTCA_0_10-0SP *Quick Start Guide***Playing individual notes**

Open Pocket MIDI



In the OutputPort Select your Metal Orchestra

Now press the keys to hear your Orchestra play

If the actuator sounds off for certain notes please see the **Tuning section preceding this*

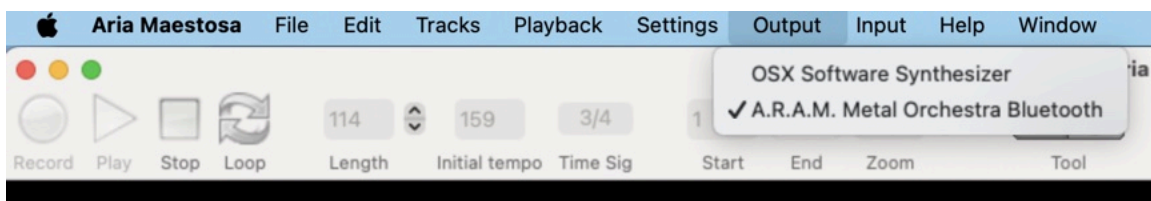


ARAM-MOTCA_0_10-OSP *Quick Start Guide*

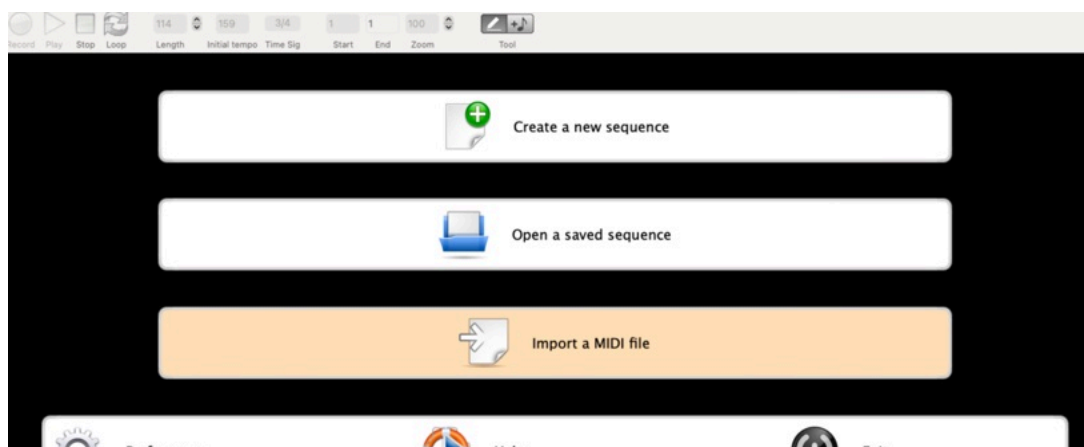
Playing Sheet Music



Open Aria Maestosa



Ensure the Metal Orchestra is selected under the Output tab



Now import any MIDI file you wish to play using the “Import a MIDI file” button

You can find tons of free sheet music online:

Simply search google or visit

<https://bitmidi.com/>

<https://github.com/ldrolez/free-midi-chords>

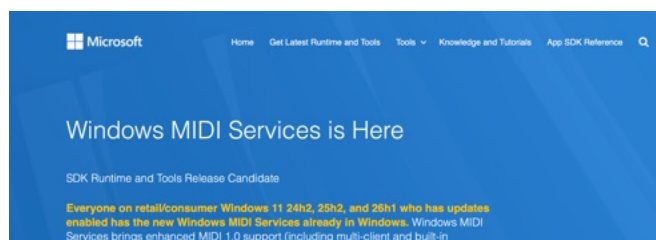


MIDI Bluetooth Operation: Using With a Windows Computer

Software	Windows MIDI services	BLE-MIDI	Pocket MIDI	Aria Maestosa
Download	https://microsoft.github.io/MIDI/get-latest/	https://apps.microsoft.com/detail/9nvmzlzttwwvl?hl=en-US&gl=US	https://apps.apple.com/us/app/pocket-midi/id1260936756?mt=12	https://ariam aestosa.github.io/arias/index.html
Use	Drivers for MIDI (May be already installed)	Connect to your instrument	Play individual notes	Play sheet music

Recommended Software

Downloading Prerequisites



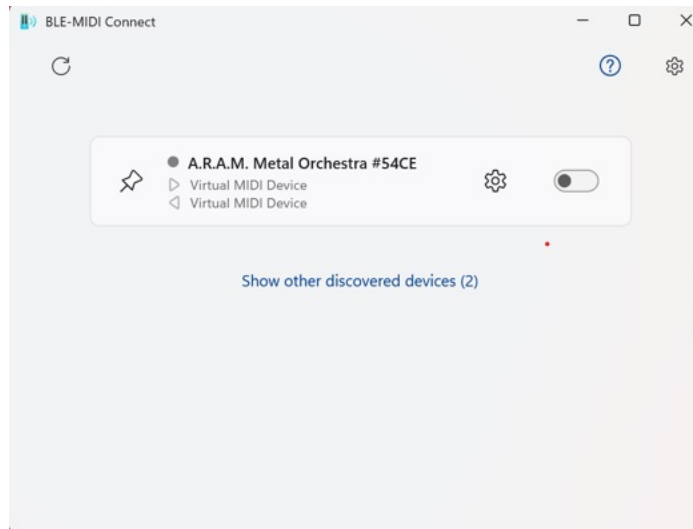
Download Windows MIDI Services SDK from the provided link:

<https://microsoft.github.io/MIDI/get-latest/>

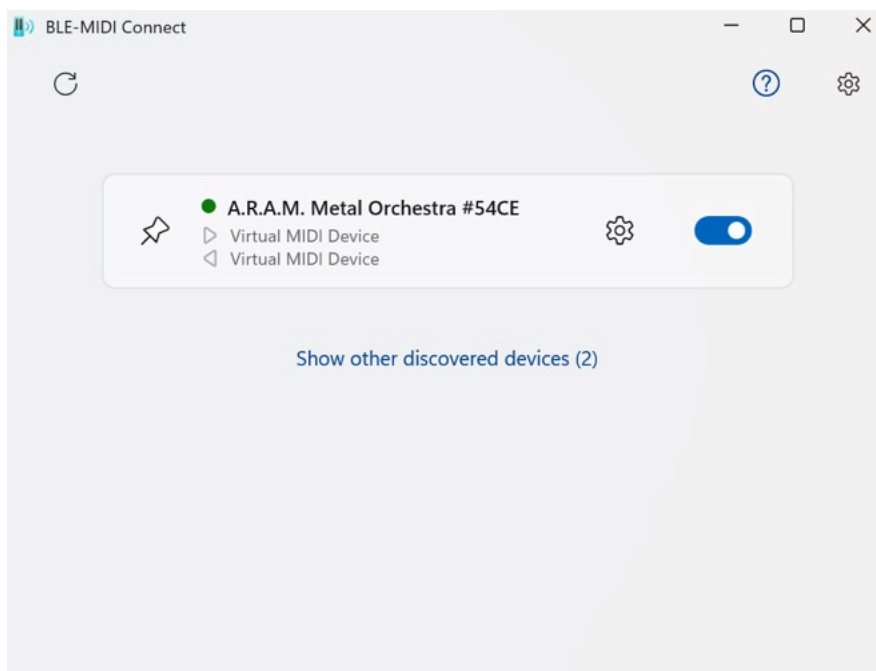
(For newer Windows 11 installs this may be already installed)



Connecting With BLE MIDI



In the main window find the device named “A.R.A.M. Metal Orchestra” and select the toggle bar



You're now successfully connected to your Metal Orchestra

**If the device does not appear, try resetting the ESP32 with the reset button*

ARAM-MOTCA_0_10-OSP *Quick Start Guide***Playing Notes With Pocket MIDI**

Open Pocket MIDI



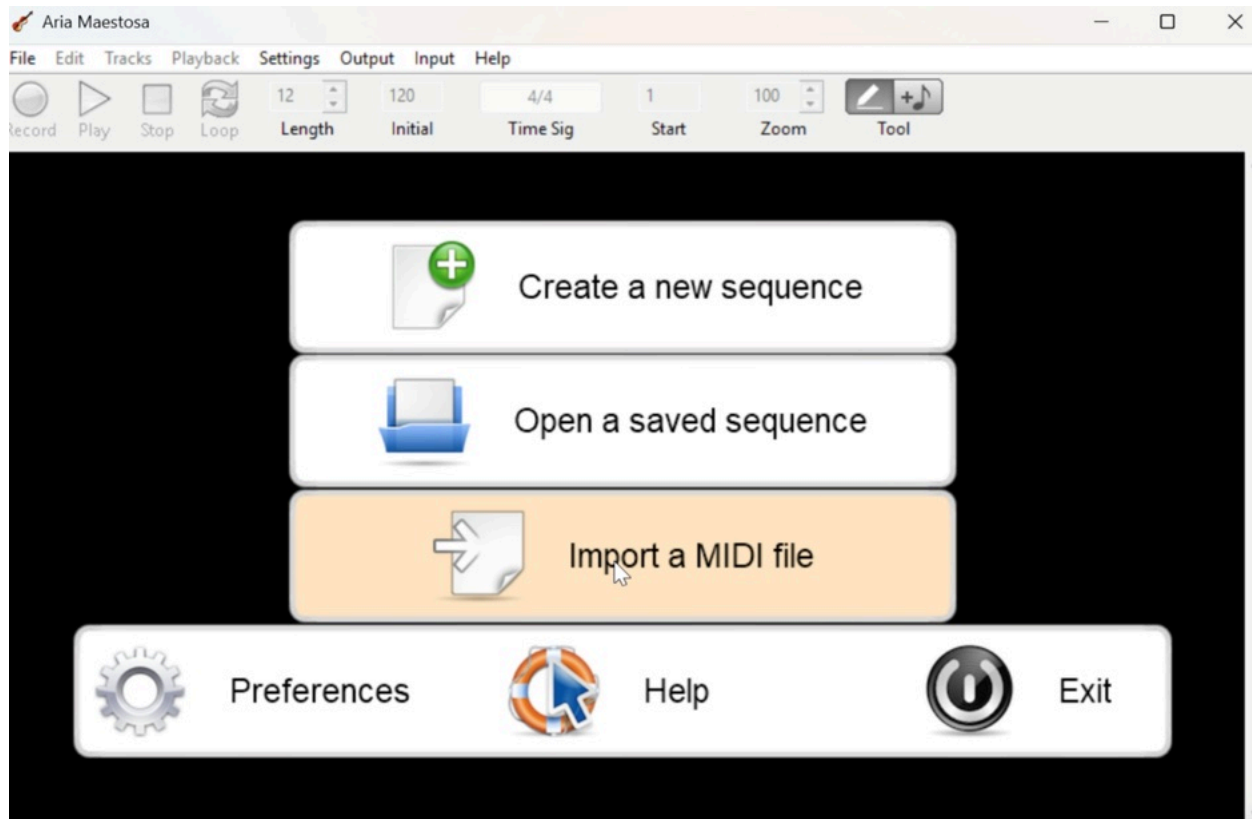
In the instrument menu select A.R.A.M. Metal Orchestra

Now simply press the keys to hear it play!

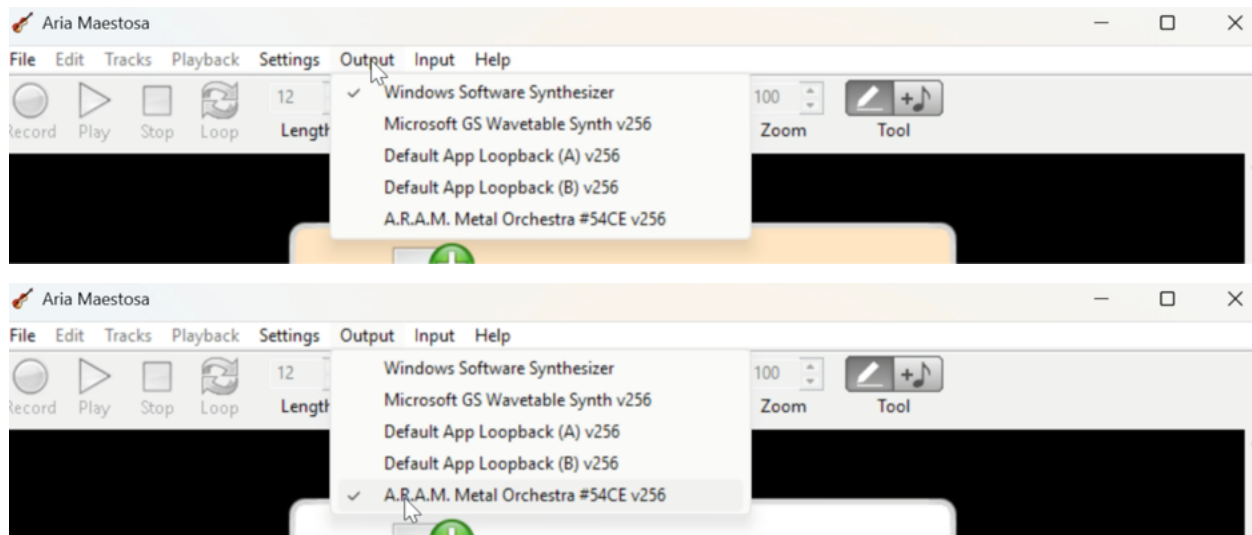
If the actuator sounds off for certain notes please see the **Tuning section preceding this*



Playing Sheet Music With Aria Maestosa



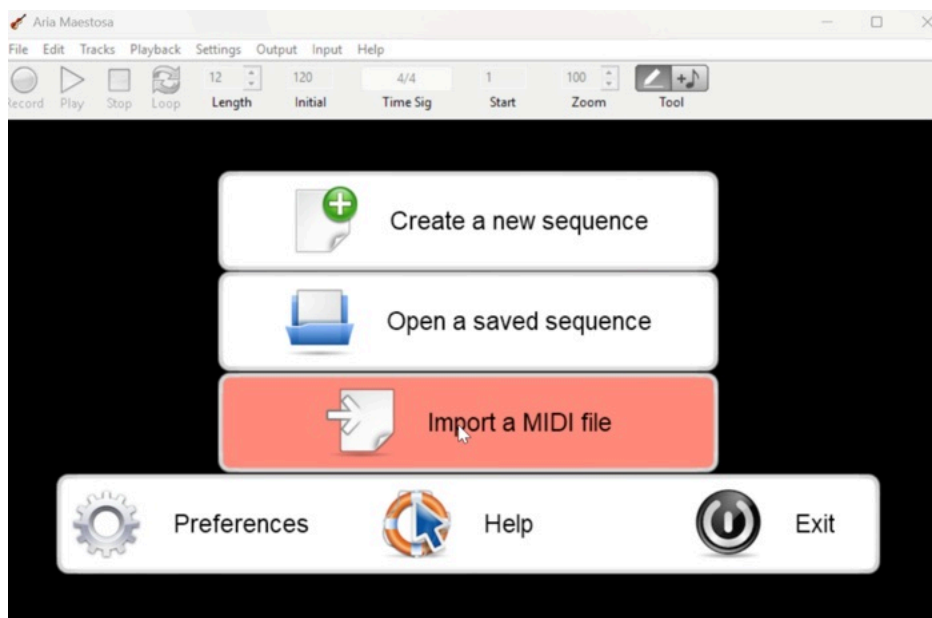
Open Aria Maestosa



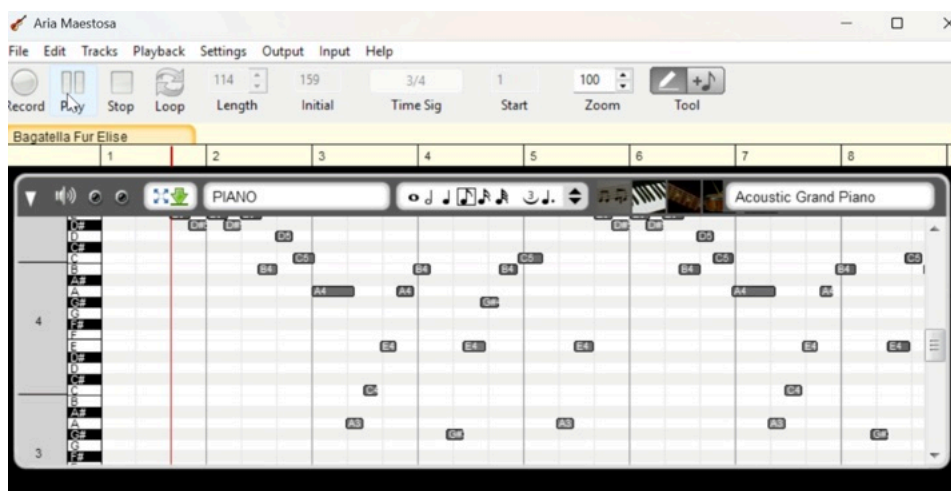
Select A.R.A.M. Metal Orchestra in the Output menu



ARAM-MOTCA_0_10-OSP *Quick Start Guide*



Now select the “Import a MIDI file” button, and in the preceding menu select your MIDI music file



Press play to hear the sheet music being played



Tuning: How to Tune Your Actuator to Almost Any Resonator

When you first setup your actuator and play notes you'll likely experience the following:

High notes sound great

Lower notes sound like a rattle rather than music

This is because we need to tune your actuator to the resonator

Software Tuning

The standard method is to do this in software

```
void setup() {  
  a1 = ARAM_TOGGLE_ACTUATOR(CNTRL_PIN_A, 800);  
  
  a1.setFrequencyDamping(400, 500, 0.5);  
  a1.setFrequencyDamping(250, 400, 0.85);  
  a1.setFrequencyDamping(200, 250, 0.75);  
  a1.setFrequencyDamping(0, 200, 0.9);  
}
```

By using

actuator.setFrequencyDamping(*minFrequency*, *maxFrequency*, *percentage*)

You can tune to certain frequencies, the percentage is the amount of damping you want to occur.

1 is 100% and the actuator will be completely silent at those frequencies

0 is 0% and the actuator will be loudest at those frequencies

Increase this at frequencies where you experience the rattling, the rattling is a result of the actuator and resonator being in too much resonance, causing a massive recoil energy.



Tuning with a magnet

Alternatively as you can also tune the actuator to the resonator by placing a magnet inside the resonator, above the actuator's plunger.

This causes the actuator to lose additional energy when it pulls against the magnet, keeping the resonance from causing recoil.

By changing the size, strength, and distance of the magnet you can tune the actuator to different frequencies.

In theory you can achieve louder notes using this method than software tuning, because the lost energy is returned on the upwards push.

Tuning with a soft barrier

You can also reduce the recoil energy by placing a soft barrier on top of the actuator, in between the magnetic feet and resonator itself. This will absorb the extra energy, eliminating the rattle also

Recommendation

It's recommended to use software tuning as this method is the easiest. However tuning with a magnet acts as a fun reinforcement of resonance, as you can hear the volume change depending on the magnet's distance as you hold above the can, it's certainly an experiment worth trying.



ARAM-MOTCA_0_10-0SP *Quick Start Guide*

DISCLAIMER – ALL PRODUCTS SOLD BY ARAM (AMERICAN ROBOTICS ASSISTED MANUFACTURING) ARE PROVIDED “AS IS” TO THE FULLEST EXTENT PERMITTED BY APPLICABLE LAW, ARAM EXPRESSLY DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. ARAM MAKES NO WARRANTY THAT THE PRODUCT WILL MEET YOUR SPECIFIC REQUIREMENTS OR THAT IT WILL OPERATE WITHOUT INTERRUPTION OR ERROR. All units are tested prior to sale; however, ARAM does not warrant that every unit will arrive in working condition. Electrical properties and specifications provided in this document are for reference purposes only. It is the sole responsibility of the user to verify that the specifications provided herein match the electrical characteristics of the unit received. Use of this product outside of stated operating parameters is at the user's own risk.

Sole Remedy – Replacement Only. ARAM takes no responsibility for any damages caused by the use of our product, whether inside or outside standard operating conditions. In the event of a defect or failure, ARAM's sole obligation, and your exclusive remedy, shall be the replacement of the defective unit within thirty (30) days of the original purchase date. To qualify for replacement, the buyer must contact ARAM within this period and is responsible for all shipping and handling costs associated with the return and replacement.

Limitation of Liability TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, IN NO EVENT SHALL ARAM BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, OR PUNITIVE DAMAGES OF ANY KIND ARISING OUT OF OR RELATED TO THE USE OR INABILITY TO USE THIS PRODUCT. ARAM'S TOTAL CUMULATIVE LIABILITY TO YOU FOR ANY AND ALL CLAIMS ARISING OUT OF OR RELATED TO THIS PRODUCT SHALL NOT EXCEED THE ORIGINAL PURCHASE PRICE PAID FOR THE SPECIFIC UNIT GIVING RISE TO THE CLAIM. THIS LIMITATION OF LIABILITY SHALL APPLY EVEN IF THE SOLE AND EXCLUSIVE REMEDY PROVIDED HEREIN IS FOUND TO HAVE FAILED OF ITS ESSENTIAL PURPOSE, AND SHALL APPLY WHETHER THE CLAIM IS BASED IN CONTRACT, TORT, STRICT LIABILITY, OR ANY OTHER LEGAL THEORY.

Important Note Some states, including New York, may have consumer protection laws that limit the enforceability of certain warranty disclaimers. Nothing in this disclaimer is intended to exclude or restrict any rights you may have under applicable state law that cannot be waived by contract.

Thank you for purchasing an ARAM product

We would like to thank you for purchasing an ARAM product. We highly value customers like you. If you have any feedback, suggestions, or questions, please reach out to our contact email. *If your unit arrives with any issues please contact us within 30 days for a replacement.* We would like to thank you for supporting ARAM, American Robotics Assisted Manufacturing, a 100% American based work force and company.

Sincerely:

Jack Serlin,

Founder of American Robotics Assisted Manufacturing

Contact Email:

Support@Aram-usa.com